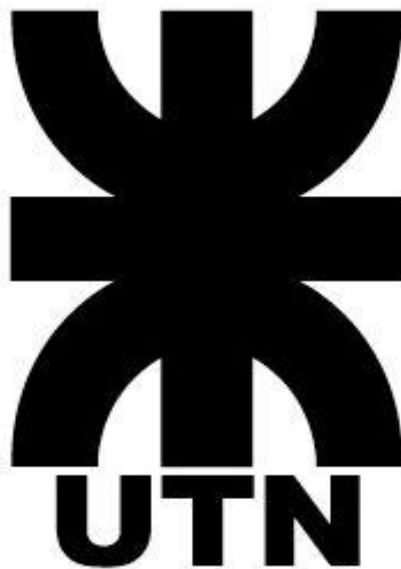


**UNIVERSIDAD TECNOLÓGICA
NACIONAL
FACULTAD REGIONAL DE TUCUMÁN**



ACTIVIDAD DE FORMACIÓN PRACTICA N°1

***Programación de microcontroladores con
STM32CubeIDE***

GRUPO 1

Integrantes:

- Pistan Ulises
- Leguizamón Marcelo
- Rivero Andrés Martin
- Lucero Dario Alejandro

Índice

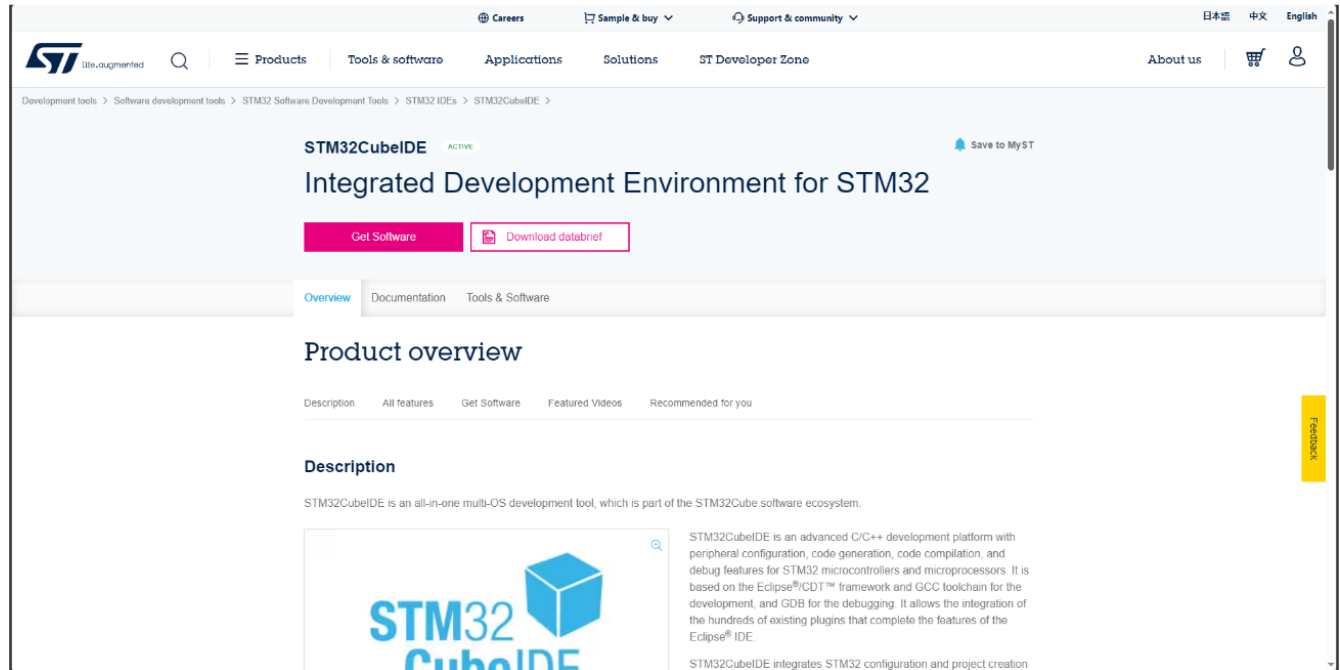
Descarga	2
Instalación	3
Creación de un Nuevo Proyecto	7
Proyecto de ejemplo: “Parpadeo de un LED”	13
Escritura del Código	13
Compilación del proyecto.....	14
Debug	16
Enlaces	19

Descarga

Para descargar el STM32CubeIDE nos dirigimos a la página oficial de STMicroelectronics:

<https://www.st.com/en/development-tools/stm32cubeide.html>

En donde observaremos lo siguiente:

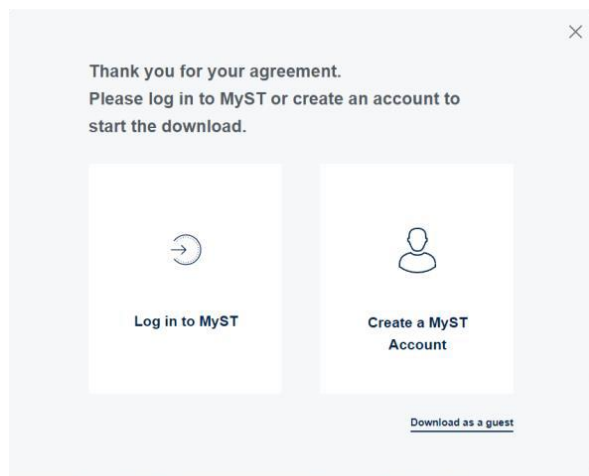


Debemos desplazarnos hasta la sección “Get Software”, donde encontraremos las distintas opciones de descarga según el sistema operativo y la versión del software que deseemos instalar. Si queremos obtener la versión más reciente disponible, simplemente hacemos clic en “Get latest”.

Get Software

	Part Number	General Description	Latest version	Download	All versions
+	STM32CubeIDE-DEB	STM32CubeIDE Debian Linux Installer	1.16.0	Get latest	Select version
+	STM32CubeIDE-Lnx	STM32CubeIDE Generic Linux Installer	1.16.0	Get latest	Select version
+	STM32CubeIDE-Mac	STM32CubeIDE macOS Installer	1.16.0	Get latest	Select version
+	STM32CubeIDE-RPM	STM32CubeIDE RPM Linux Installer	1.16.0	Get latest	Select version
+	STM32CubeIDE-Win	STM32CubeIDE Windows Installer	1.16.0	Get latest	Select version

Para poder descargar el IDE, el sitio web nos pedirá que creamos una cuenta. En caso de ya tener una, deberemos iniciar sesión. Este paso es obligatorio para acceder a la descarga del software.

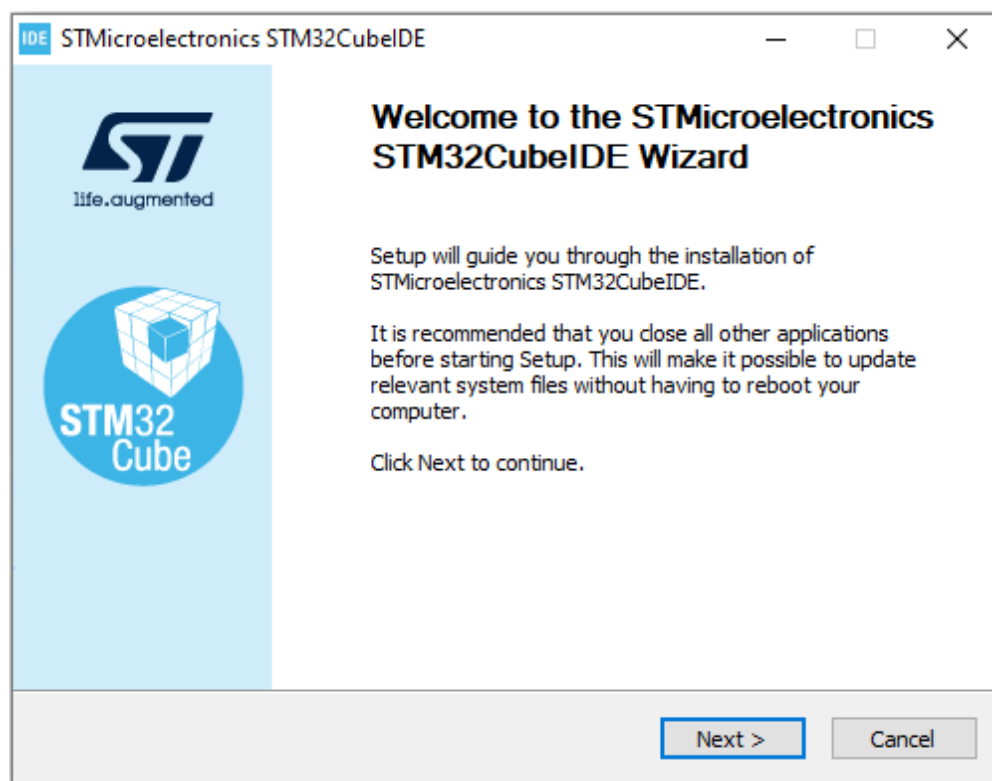


Una vez creada la cuenta, nos permitirá descargar el instalador.

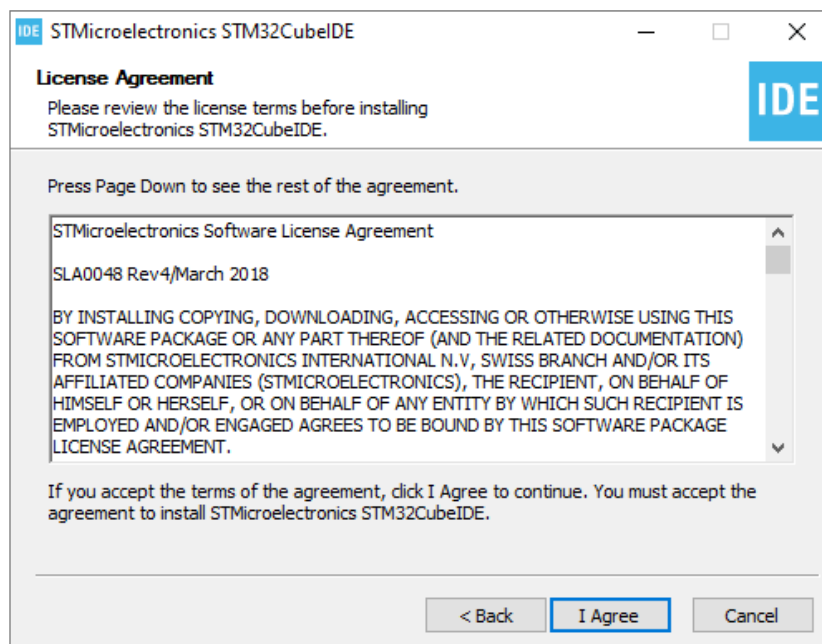
Instalación

Iniciamos la instalación ejecutando el archivo del instalador que se encuentra en la carpeta de Descargas.

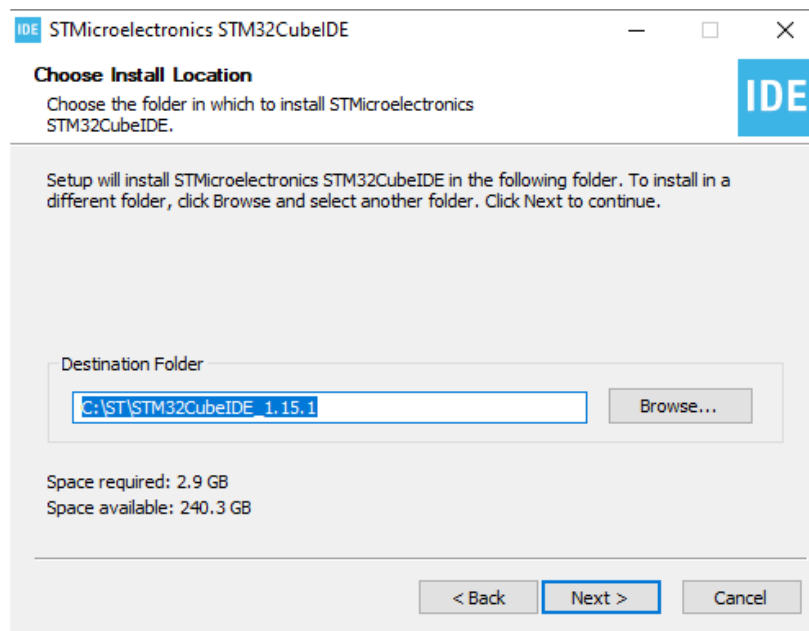
Al hacerlo, se abrirá la siguiente ventana:



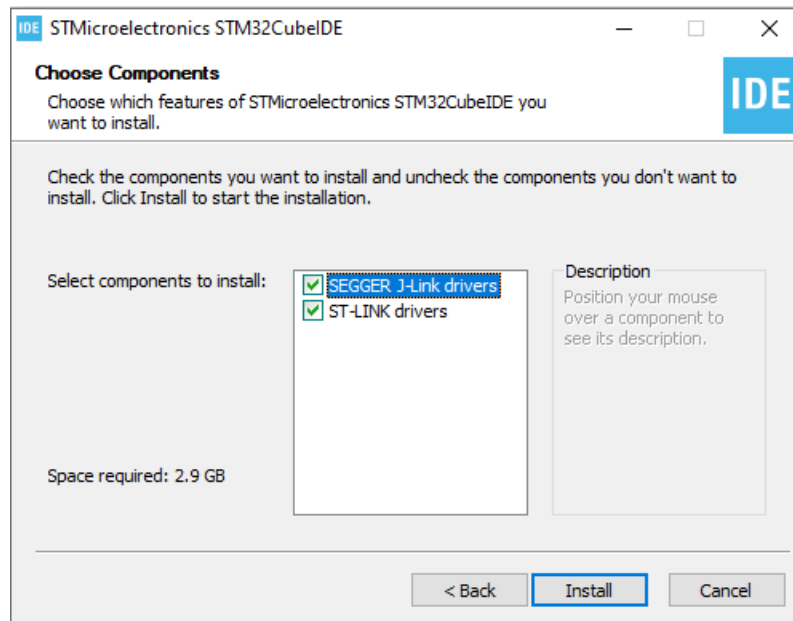
Hacemos clic en “Next >” y se mostrará la siguiente pantalla:



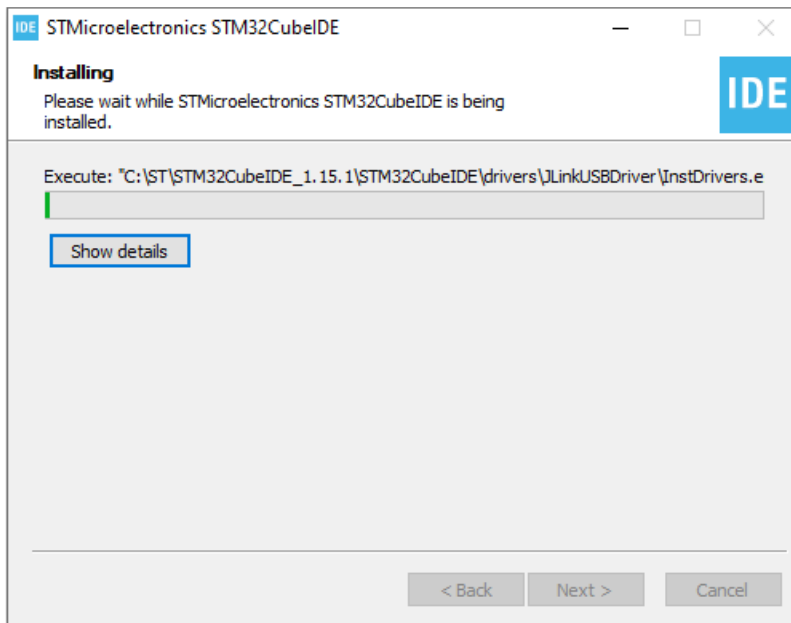
Aceptamos los términos de uso haciendo clic en “I Agree”. A continuación, se mostrará un menú que nos permite seleccionar la carpeta de destino donde se instalará el software.



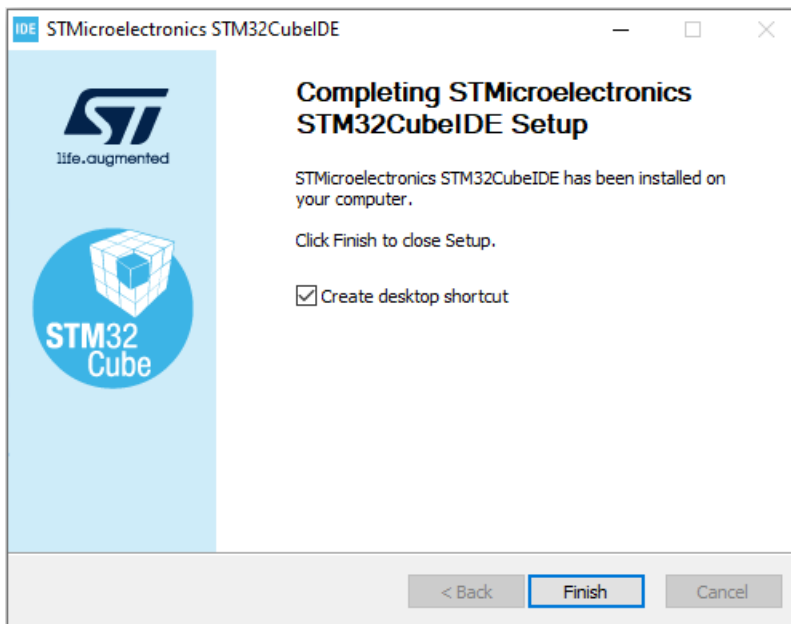
Hacemos clic en “Next >” y se mostrará un nuevo menú que nos permite seleccionar la instalación de complementos adicionales.



Presionamos en “Install” y comenzara el proceso de instalación.



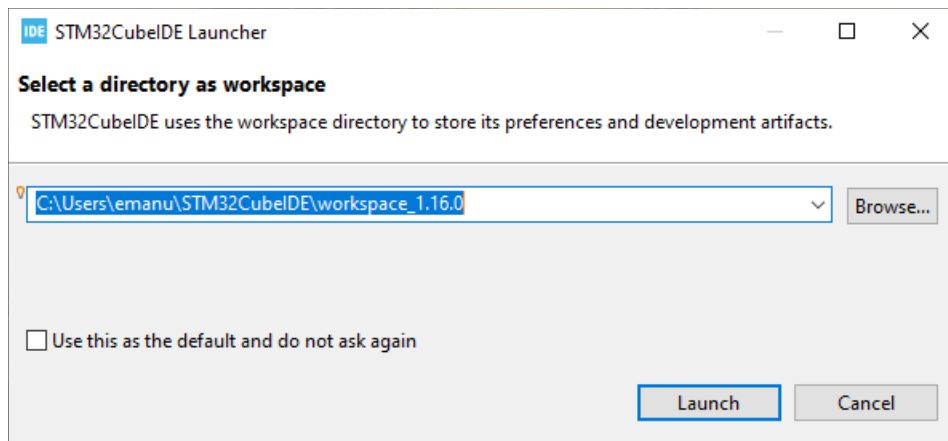
Una vez finalizada la instalación del IDE, se mostrará la siguiente pantalla:



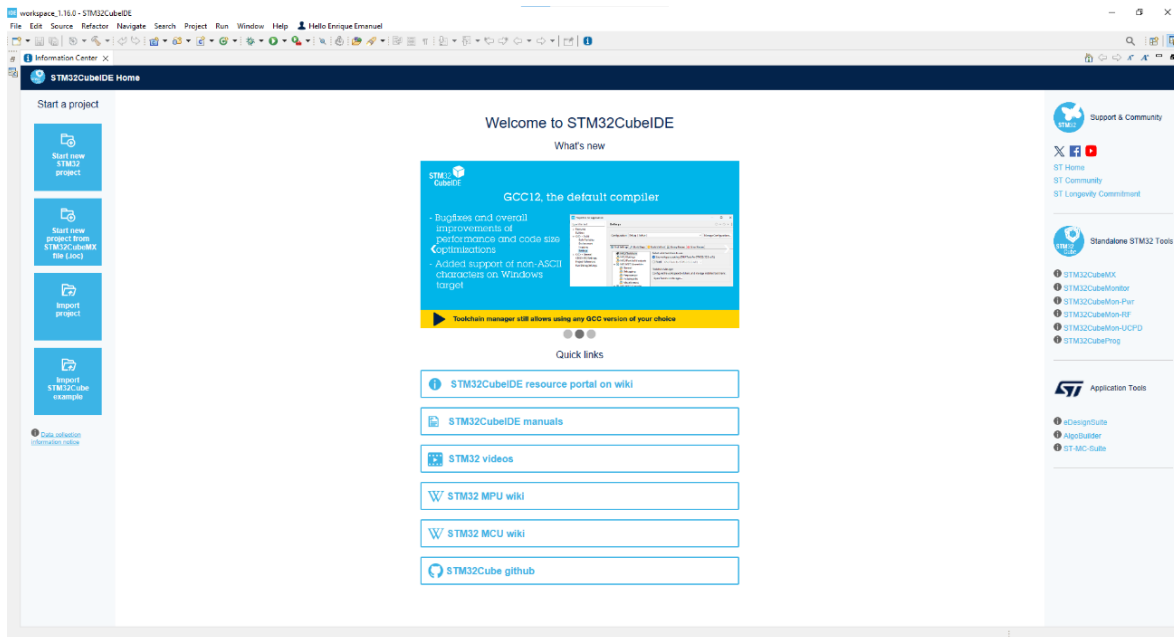
Si deseamos crear un acceso directo en el escritorio para abrir el IDE, marcamos la casilla "Create desktop shortcut". Luego, hacemos clic en "Finish" para finalizar el proceso de instalación.

Creación de un Nuevo Proyecto

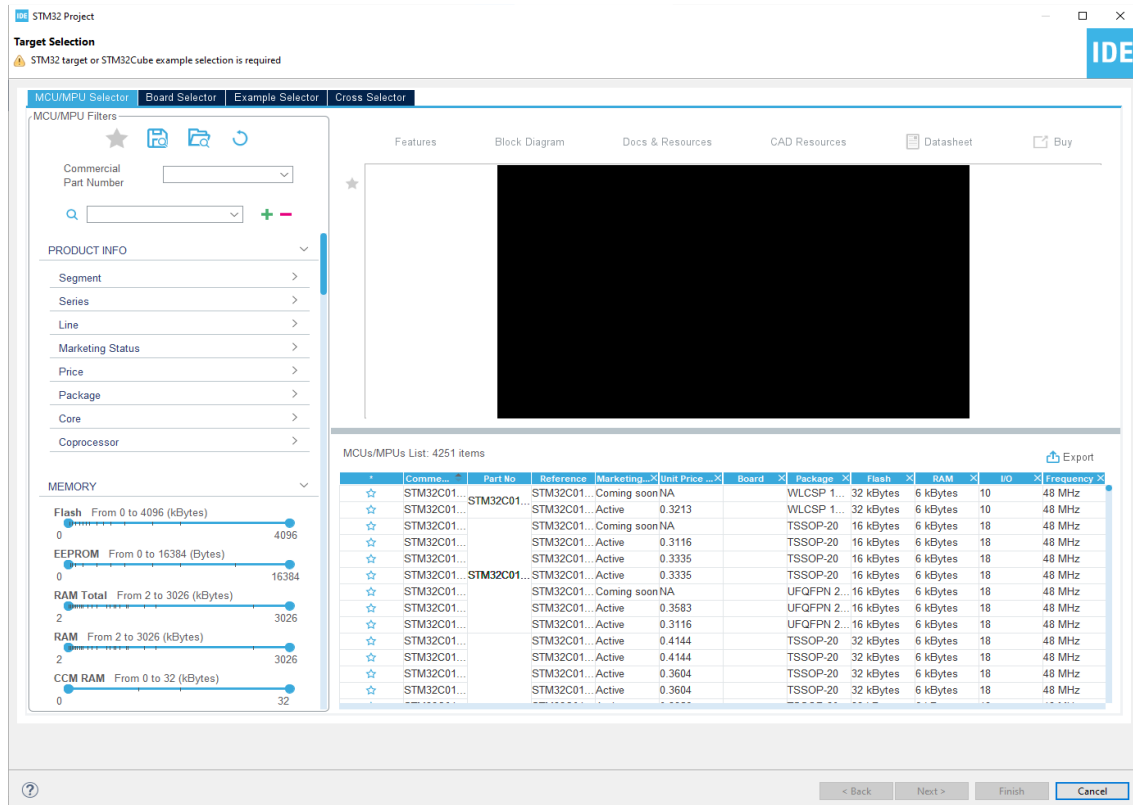
Al abrir el IDE por primera vez, se nos pedirá especificar el *workspace* (espacio de trabajo), que es la ubicación donde se guardarán nuestros proyectos.



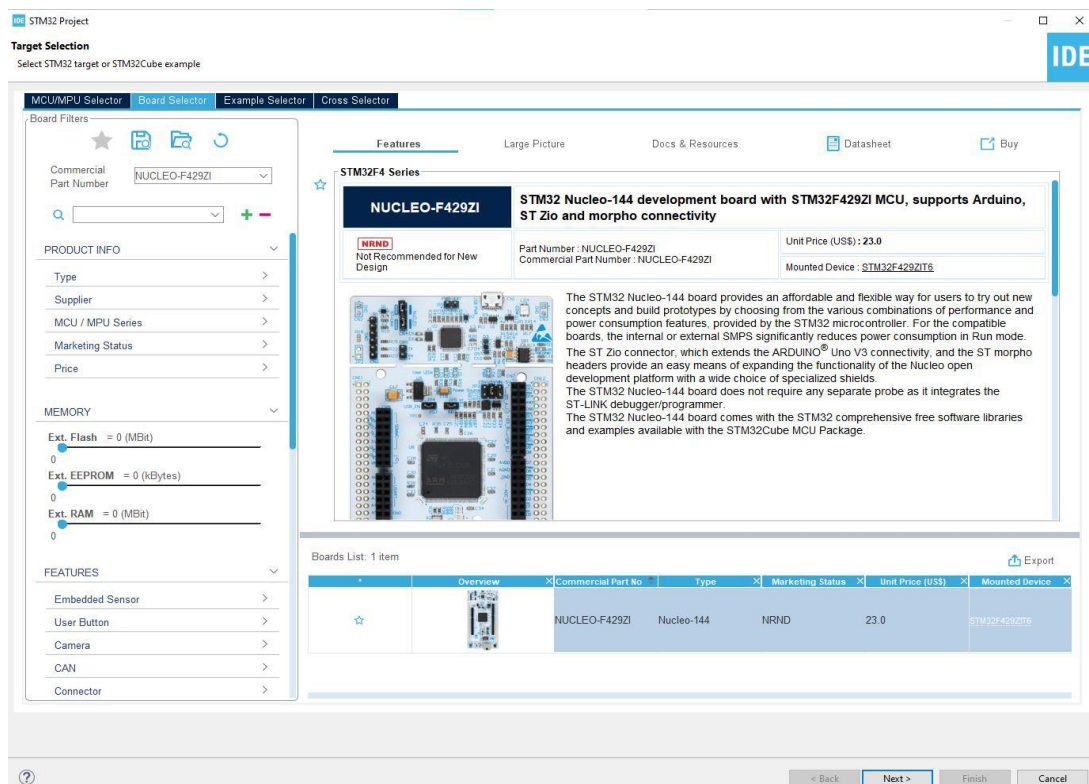
Para comenzar, seleccionamos una carpeta y hacemos clic en 'Launch'. En la página de inicio, se presentan varias opciones, como iniciar un nuevo proyecto o importar uno existente. Además, se incluyen enlaces a videos, manuales y documentación sobre el IDE, que facilitan el aprendizaje y uso de la herramienta.



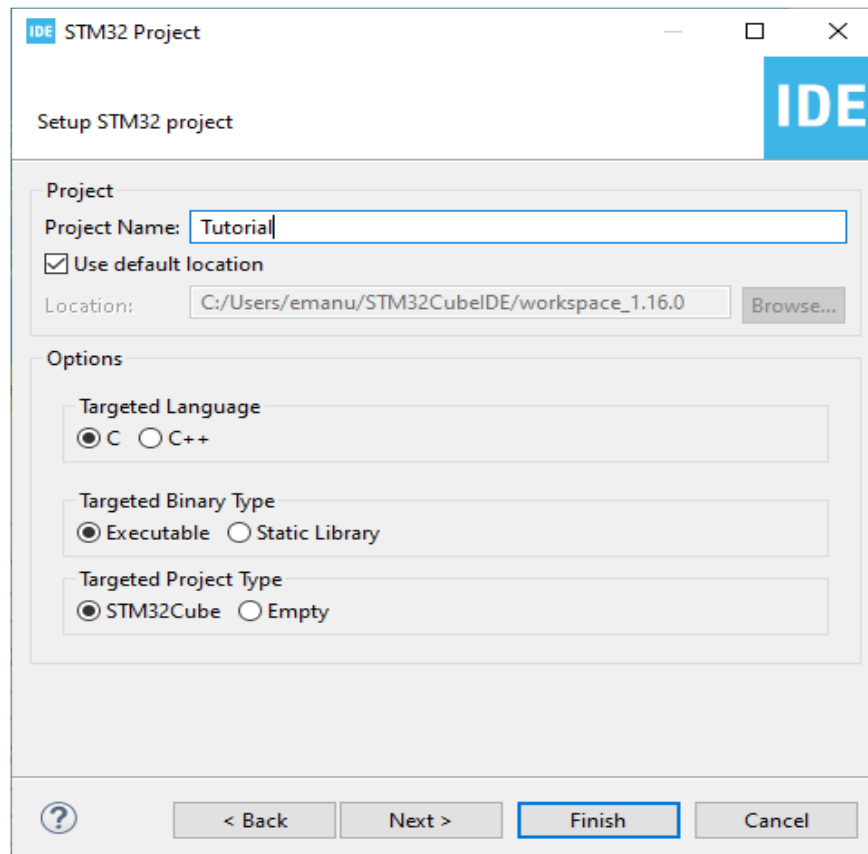
Para crear un nuevo proyecto, hacemos clic en “Start new STM32 project”.



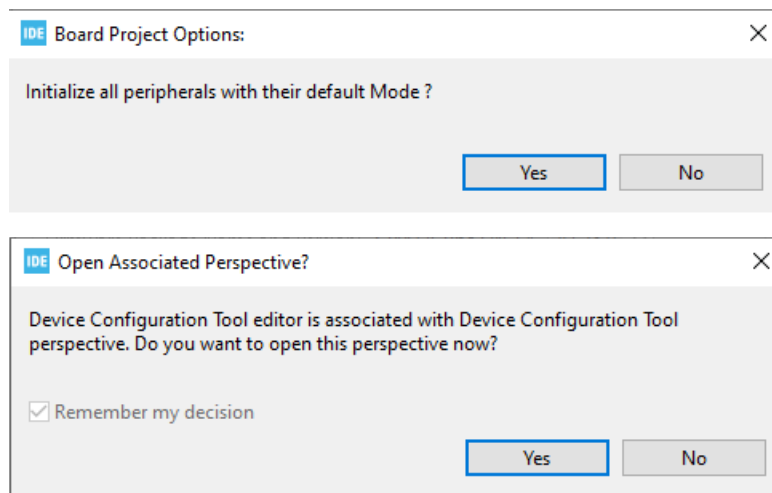
En esta ventana, seleccionamos el microcontrolador o la placa de desarrollo que utilizaremos para el proyecto.



Tras seleccionar la placa de desarrollo, procedemos a asignar un nombre al proyecto. A continuación, hacemos clic en 'Finish'. Esto abrirá una nueva ventana donde debemos confirmar ambas opciones presentadas.

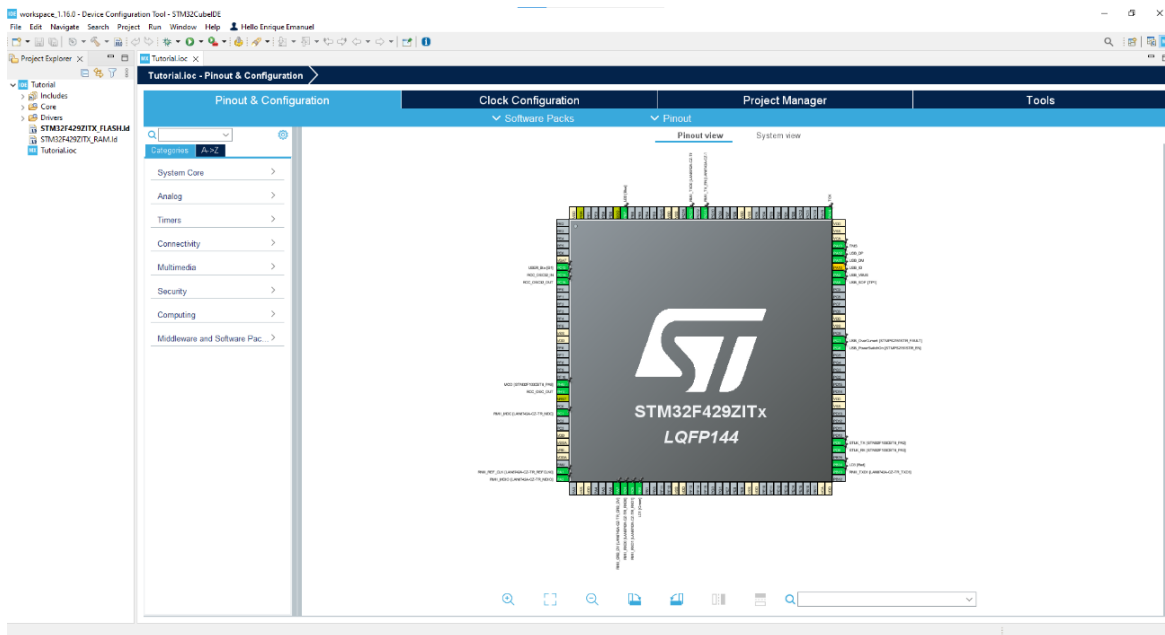


Hacemos clic en "Finish". A continuación, aparecerá una nueva ventana:

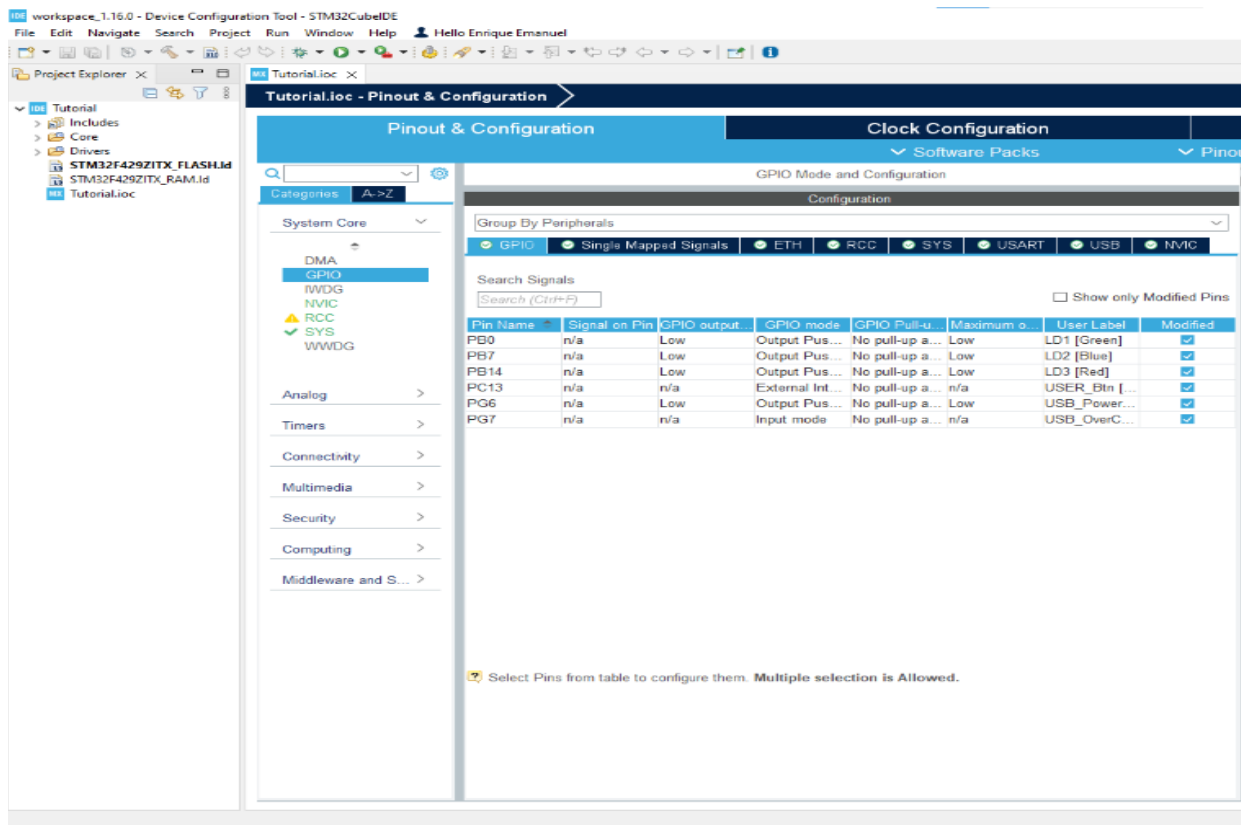


Una vez confirmadas ambas ventanas, se abrirá automáticamente el archivo con extensión '.ioc'. En STM32CubeIDE, un archivo '.ioc' (Input Output Configuration) es un archivo de configuración que especifica los periféricos, pines y ajustes de hardware para un proyecto específico de STM32. Este archivo es generado por STM32CubeMX, una herramienta gráfica de configuración integrada en STM32CubeIDE.

El archivo .ioc permite al usuario configurar los pines del microcontrolador, así como habilitar y personalizar periféricos como UART, SPI, I2C, ADC, entre otros. Además, facilita la configuración del sistema de reloj del microcontrolador, incluyendo la selección de la fuente de reloj y las divisiones necesarias para obtener las frecuencias deseadas.

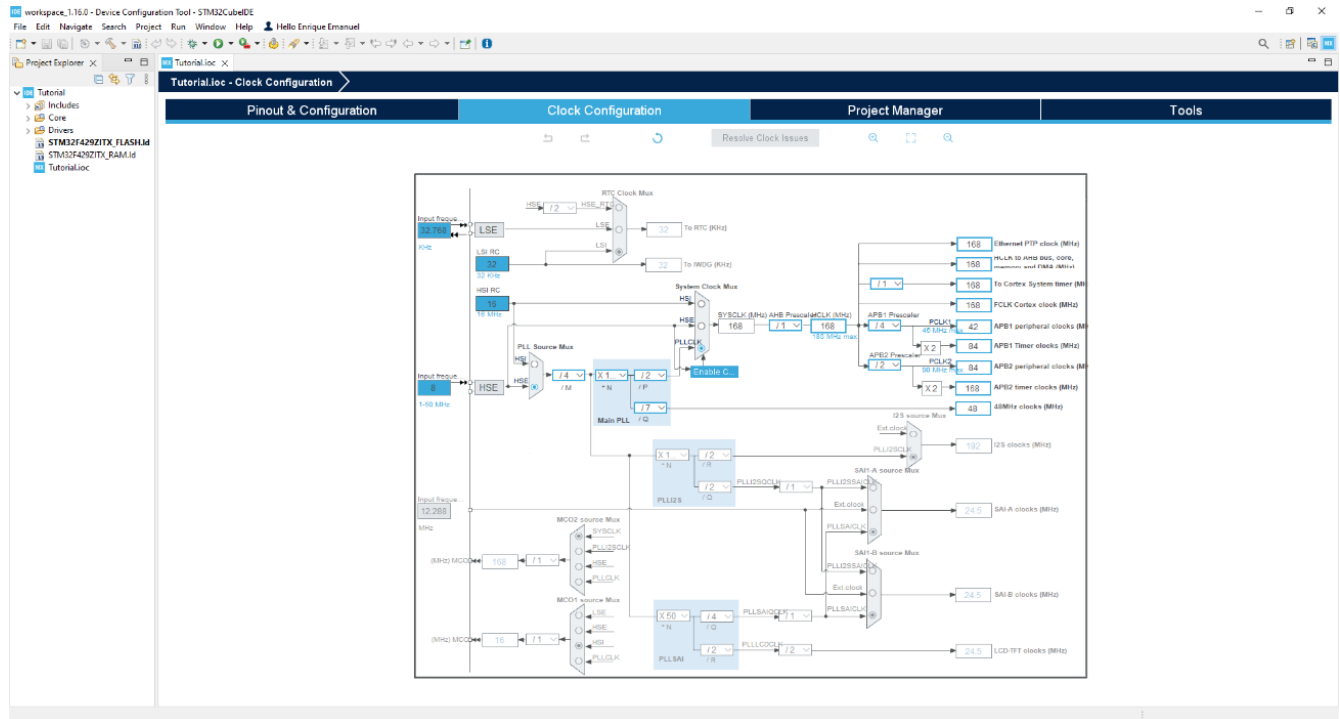


Todas las configuraciones están accesibles en el panel lateral de la interfaz.

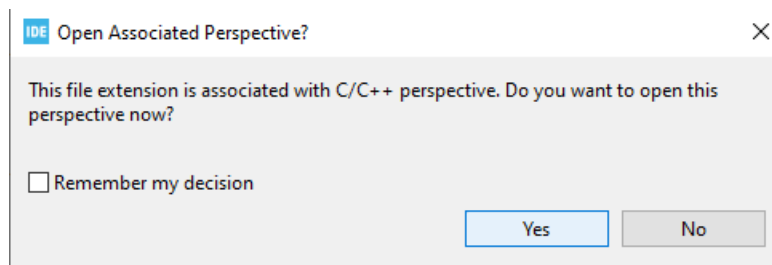


Al seleccionar correctamente la placa de desarrollo, el software asigna automáticamente los pines correspondientes a los LEDs y botones de la placa. Si esto no ocurre o si el proyecto requiere configuraciones adicionales no incluidas por defecto, estas se pueden realizar en el panel lateral. Se recomienda completar todas las configuraciones antes de comenzar a escribir el código y evitar modificarlas durante el proceso. Esto se debe a que, una vez finalizada la configuración, el archivo '.ioc' genera automáticamente el código inicial de configuración y los archivos de cabecera necesarios para el proyecto, incluyendo la inicialización de periféricos y la configuración de pines según lo especificado.

Como paso siguiente, se debe configurar la frecuencia del reloj. Si la placa de desarrollo se ha seleccionado adecuadamente, esta configuración se establece correctamente por defecto.



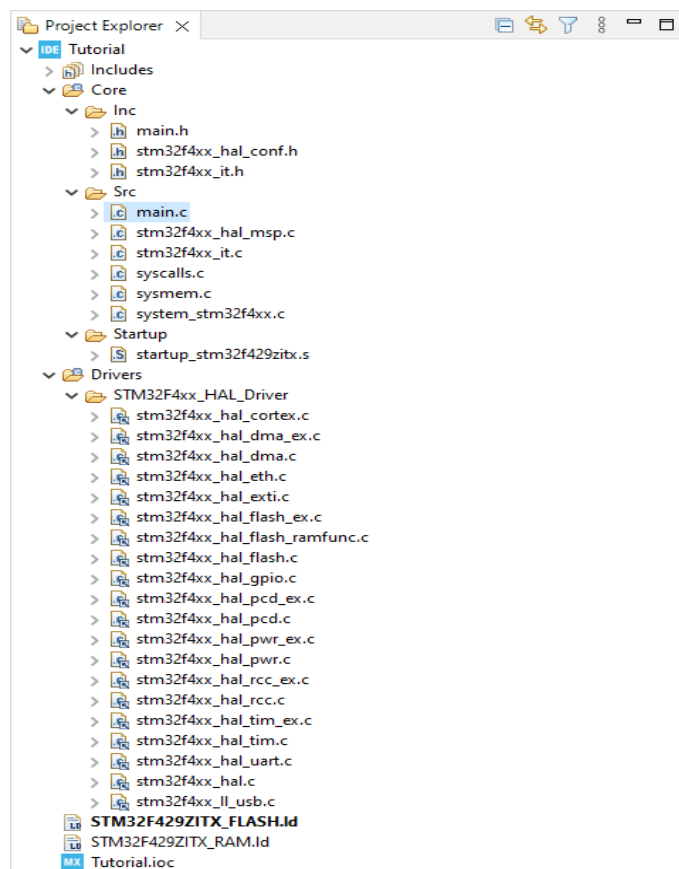
Al guardar la configuración del archivo .ioc, se abrirá la siguiente ventana:



Al confirmar esta ventana se genera el código correspondiente.

Al guardar la configuración del archivo .ioc, se generan automáticamente varias carpetas con los siguientes contenidos:

- **Includes:** Carpeta configurada como ruta de inclusión para los archivos de cabecera (.h), que contienen definiciones esenciales para el proyecto.
- **Core:** Contiene archivos fundamentales para el núcleo del proyecto, necesarios para el arranque y el funcionamiento del microcontrolador.
 - **Src:** Incluye los archivos fuente (.c), como el código de inicialización del sistema, el archivo main.c y otros archivos que gestionan el núcleo del sistema.
 - **Inc:** Almacena los archivos de cabecera (.h) que definen funciones, macros y tipos de datos esenciales para la configuración y el arranque del microcontrolador. Entre ellos, se encuentra el archivo main.h, que incluye las etiquetas de los pines configurados en el archivo .ioc, entre otras definiciones.
- **Drivers:** Contiene los controladores de hardware necesarios para interactuar con los periféricos del microcontrolador.
 - **HAL:** Incluye los controladores HAL (Hardware Abstraction Layer) generados para los periféricos configurados en el archivo .ioc. La HAL proporciona una capa de abstracción que permite interactuar con los periféricos de forma independiente del hardware específico, mejorando la portabilidad del código.

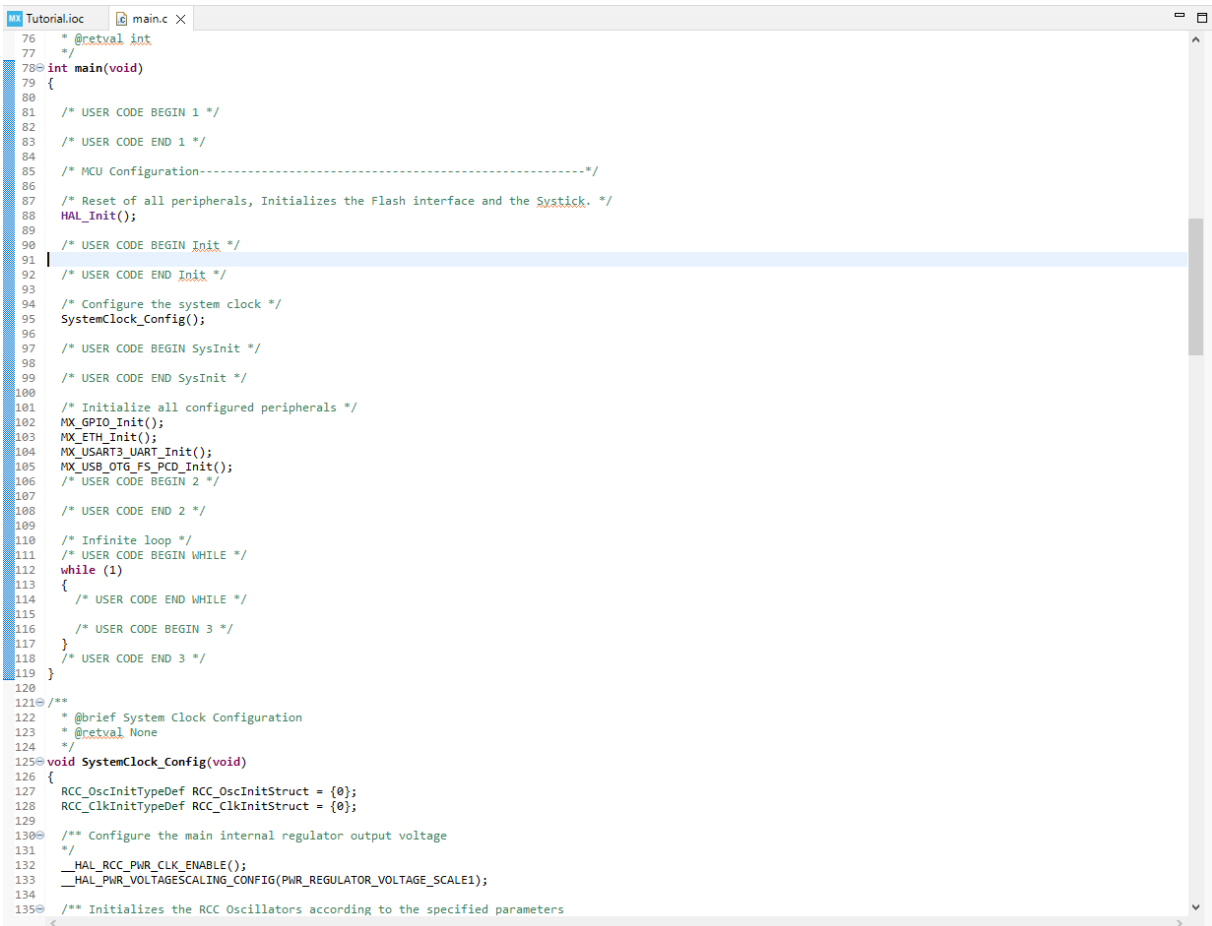


Proyecto de Ejemplo: “Parpadeo de un LED”

A continuación, desarrollaremos un ejemplo práctico de un proyecto: una aplicación que hace parpadear un LED integrado en la placa de desarrollo cada 250 milisegundos.

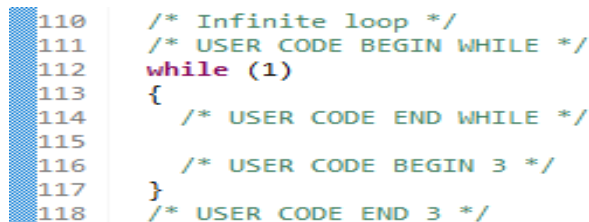
Escritura del Código

Una vez configurado correctamente el archivo .ioc y asignada una etiqueta al pin correspondiente al LED integrado en la placa, se genera el código automáticamente. Para continuar, abrimos el archivo main.c, ubicado en la ruta /Core/Src/main.c. El código generado tendrá la siguiente estructura inicial:



```
76  * @retval int
77  */
78  int main(void)
79  {
80
81  /* USER CODE BEGIN 1 */
82
83  /* USER CODE END 1 */
84
85  /* MCU Configuration-----*/
86
87  /* Reset of all peripherals, Initializes the Flash interface and the SysTick. */
88  HAL_Init();
89
90  /* USER CODE BEGIN Init */
91
92  /* USER CODE END Init */
93
94  /* Configure the system clock */
95  SystemClock_Config();
96
97  /* USER CODE BEGIN SysInit */
98
99  /* USER CODE END SysInit */
100
101  /* Initialize all configured peripherals */
102  MX_GPIO_Init();
103  MX_ETH_Init();
104  MX_USART3_UART_Init();
105  MX_USB_OTG_FS_PCD_Init();
106  /* USER CODE BEGIN 2 */
107
108  /* USER CODE END 2 */
109
110  /* Infinite loop */
111  /* USER CODE BEGIN WHILE */
112  while (1)
113  {
114      /* USER CODE END WHILE */
115
116      /* USER CODE BEGIN 3 */
117  }
118  /* USER CODE END 3 */
119  }
120
121  /**
122   * @brief System Clock Configuration
123   * @retval None
124   */
125  void SystemClock_Config(void)
126  {
127      RCC_OscInitTypeDef RCC_OscInitStruct = {0};
128      RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
129
130      /** Configure the main internal regulator output voltage
131       */
132      __HAL_RCC_PWR_CLK_ENABLE();
133      __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE1);
134
135      /** Initializes the RCC Oscillators according to the specified parameters
```

La aplicación se inicia con la función main, la cual contiene un bucle infinito (while(1)) donde se implementa la lógica principal del programa.



```
110      /* Infinite loop */
111      /* USER CODE BEGIN WHILE */
112      while (1)
113      {
114          /* USER CODE END WHILE */
115
116          /* USER CODE BEGIN 3 */
117      }
118      /* USER CODE END 3 */
```

El código dentro de este bucle infinito (while(1)) se ejecutará de manera continua hasta que ocurra una interrupción o se interrumpa la alimentación de la placa de desarrollo.

Dentro del bucle infinito (while(1)), implementaremos el código de nuestra aplicación, ya que este se ejecutará de manera continua mientras no se produzca una interrupción o se corte la alimentación de la placa de desarrollo. Se recomienda escribir el código entre los comentarios:

```
/* USER CODE BEGIN */
```

```
/* USER CODE END */
```

Esto asegura que las modificaciones realizadas se conserven si se edita el archivo .ioc, ya que cualquier cambio en este archivo genera un nuevo código base, sobrescribiendo las secciones fuera de estos comentarios.

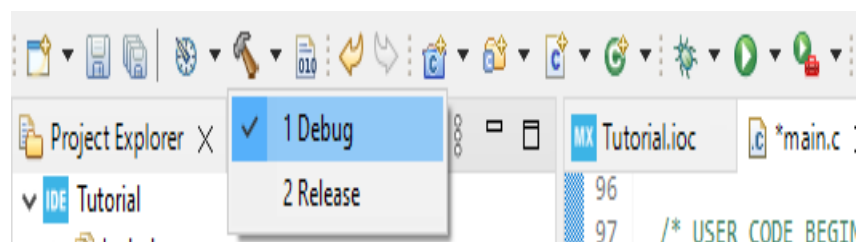
Para controlar el parpadeo del LED, utilizaremos dos funciones de la HAL (Hardware Abstraction Layer):

1. **HAL_GPIO_TogglePin:** Esta función, perteneciente al módulo GPIO (General Purpose Input Output), invierte el estado de una salida digital. Si el pin está en estado alto, pasa a estado bajo, y viceversa. Requiere dos parámetros: el puerto (por ejemplo, GPIOA) y el pin (por ejemplo, GPIO_PIN_5).
2. **HAL_Delay:** Esta función introduce un retardo en la ejecución del programa, pausando el código durante un número específico de milisegundos antes de continuar. Toma como parámetro la cantidad de milisegundos del retardo.

```
110  /* Infinite loop */
111  /* USER CODE BEGIN WHILE */
112  while (1)
113  {
114      HAL_GPIO_TogglePin(LD1_GPIO_Port, LD1_Pin);
115      HAL_Delay(250);
116
117      /* USER CODE END WHILE */
118
119      /* USER CODE BEGIN 3 */
120  }
121  /* USER CODE END 3 */
122 }
```

Compilación del proyecto

Para compilar el proyecto, dirígete al ícono del martillo en el menú superior y selecciona las opciones desplegables.



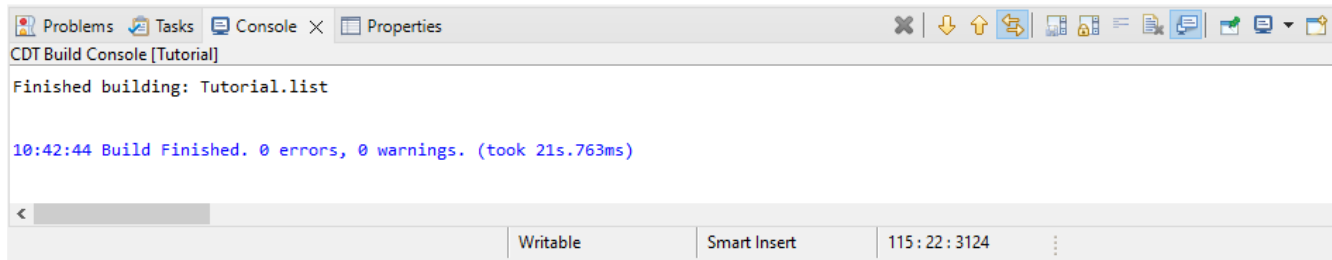
El modo “Debug” está diseñado para optimizar la depuración del programa.

Modo Release y compilación

El modo “Release” está optimizado para la ejecución final del programa en un entorno de producción.

Al hacer clic en el ícono del martillo, se inicia el proceso de compilación del programa.

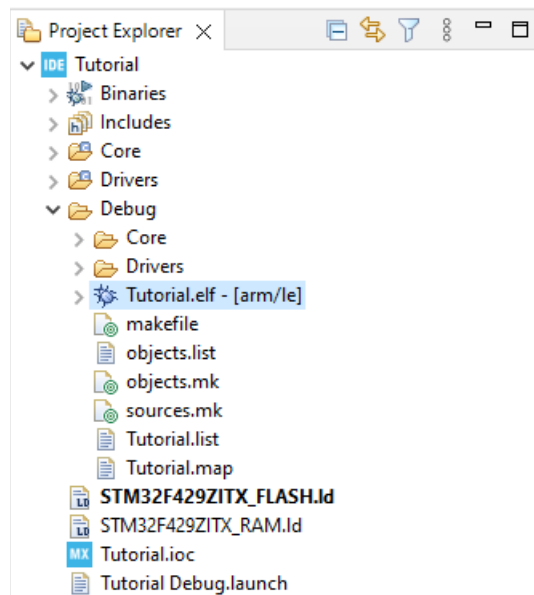
Una vez finalizada la compilación, en la parte inferior de la interfaz se mostrará la consola, donde aparecerá la siguiente información:



Gestión de errores y resultado de la compilación

Si la compilación falla debido a un error en el código, la consola indicará la línea donde se encuentra el error junto con una breve descripción del problema. Es necesario corregir todos los errores de sintaxis para que el programa pueda compilarse correctamente.

Una vez completada la compilación, se genera un archivo con extensión “.elf” en la carpeta “Debug”.

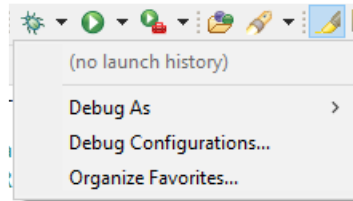


El archivo con extensión “.elf” contiene el código máquina que ejecutará el microcontrolador, además de información adicional como datos de depuración, secciones de datos y metadatos del programa.

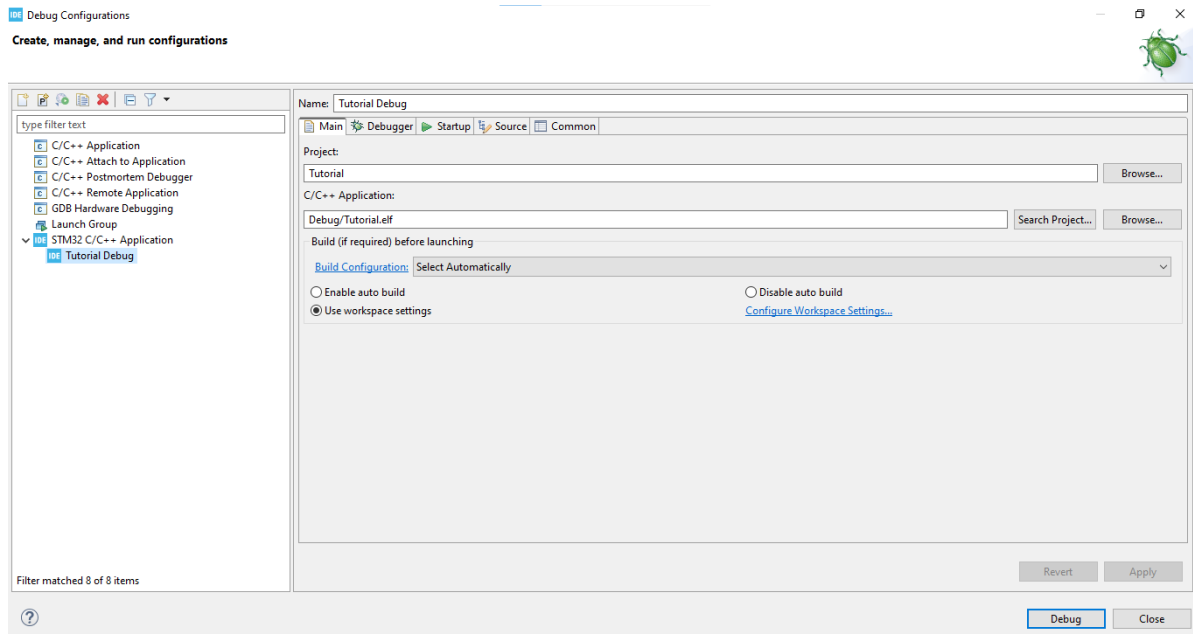
Depuración del programa

El propósito principal de la depuración es asegurar que el programa funcione correctamente, identificando y corrigiendo errores lógicos, fallos de ejecución o problemas de rendimiento que puedan afectar su funcionalidad.

Para depurar el programa, dirígete al ícono “Debug” en el menú superior, despliega las opciones y selecciona “Debug Configurations...”.

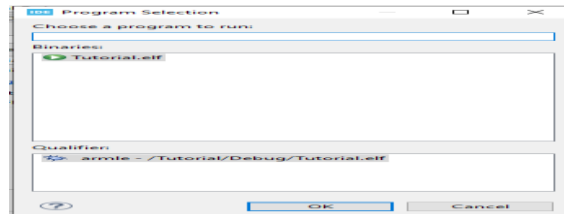


Se nos muestra la siguiente ventana:



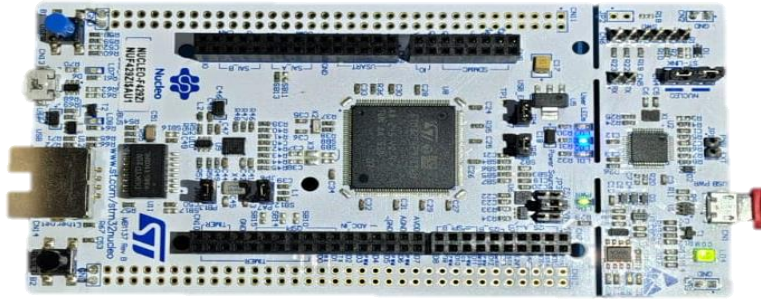
Configuración de la depuración

En la ventana de configuración, selecciona “STM32 C/C++ Application” y elige la aplicación recientemente programada. A continuación, haz clic en “Search Project...” y selecciona el archivo “.elf” generado durante el proceso de compilación.

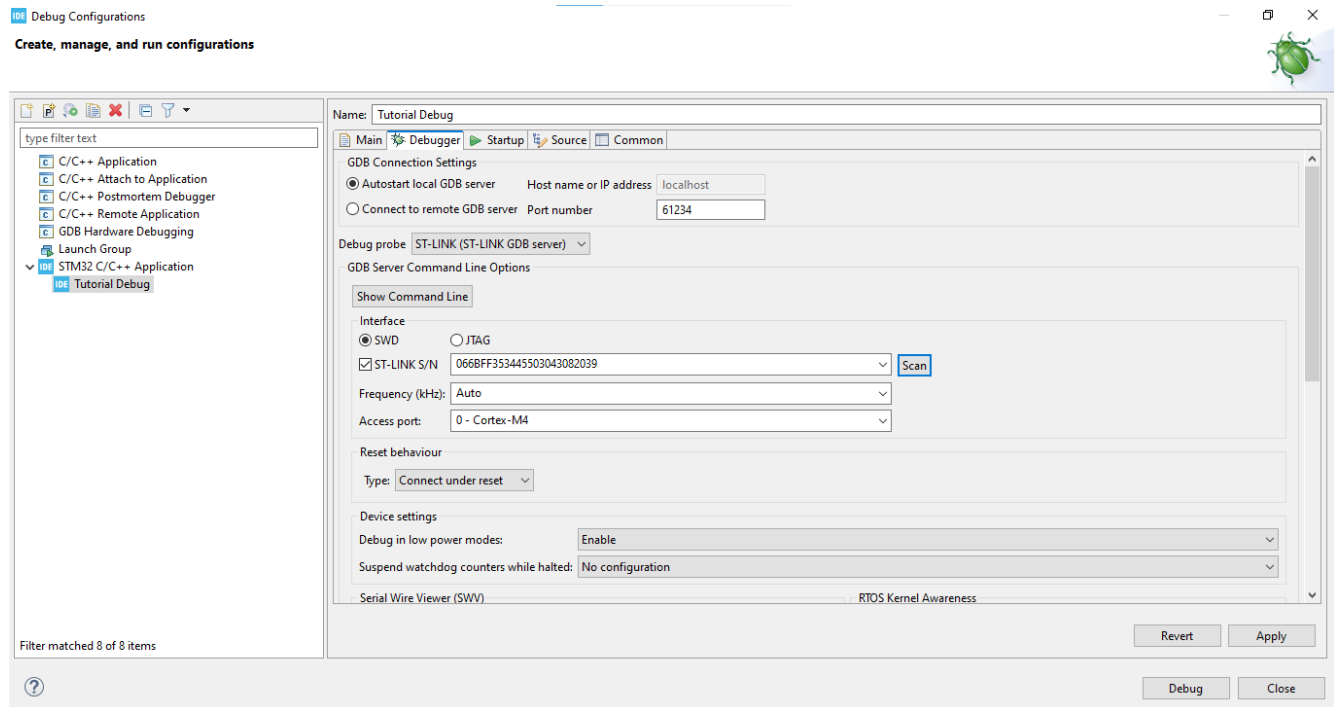


En la pestaña “Debugger”, configura el ST-LINK, una herramienta de STMicroelectronics diseñada para programar y depurar microcontroladores STM32. Esta permite cargar código en el microcontrolador y realizar la depuración de aplicaciones durante el desarrollo.

A continuación, conecta la tarjeta de desarrollo a la PC.



Luego en la pestaña “Debugger” presionamos en el boton “Scan” para que se genere la conexión.



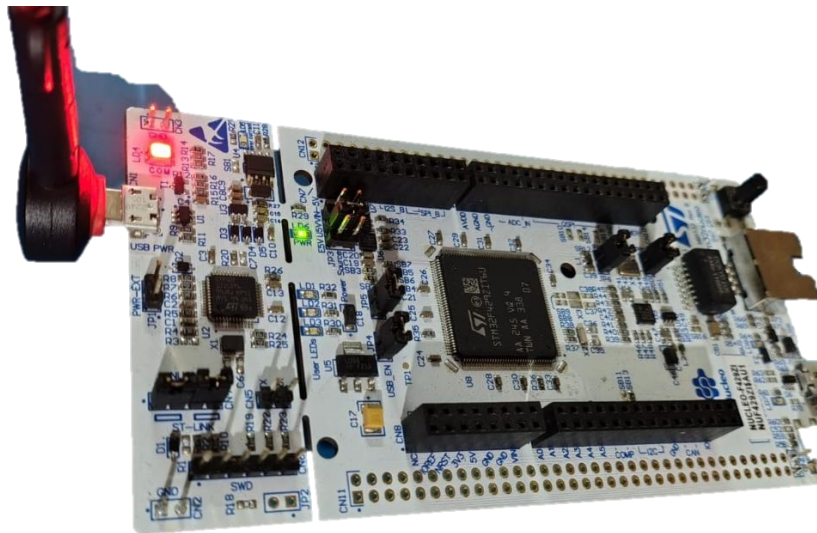
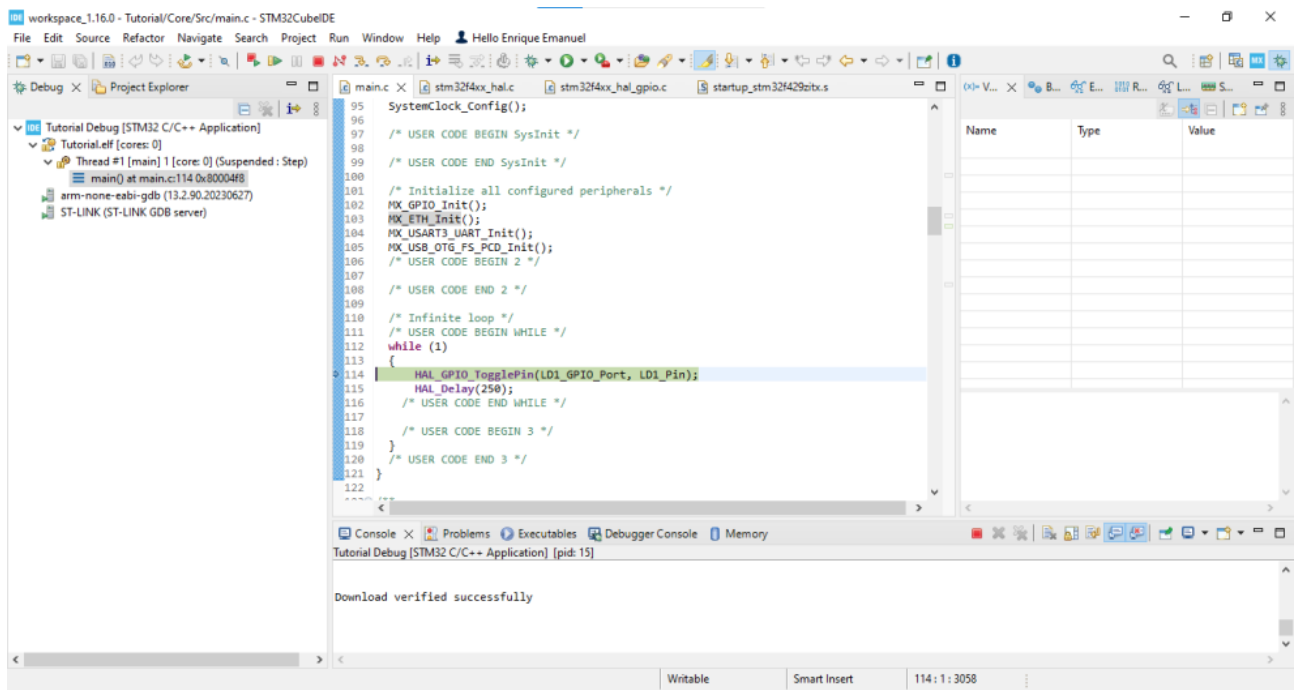
Inicio de la depuración

Una vez conectada la tarjeta de desarrollo, haz clic en “Apply” y luego en “Debug” para iniciar la depuración del código.

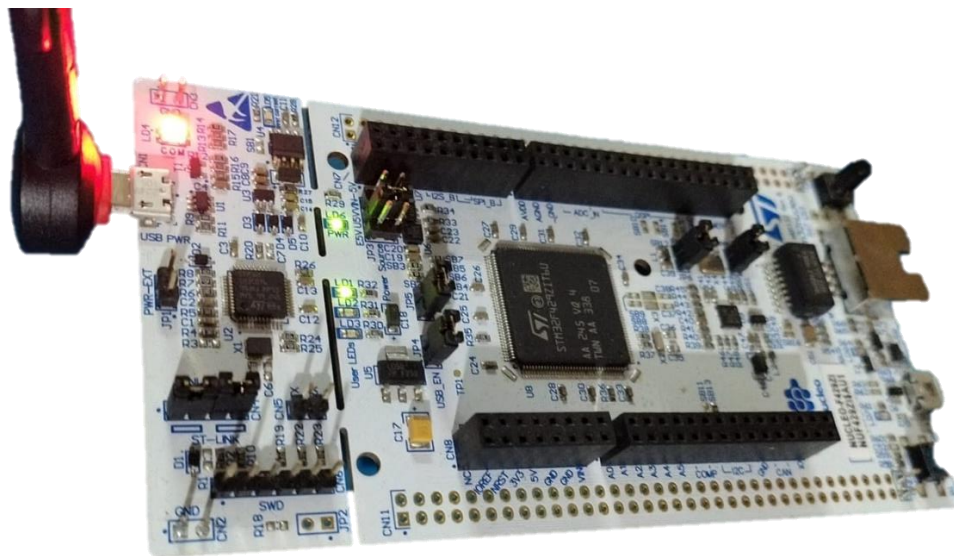
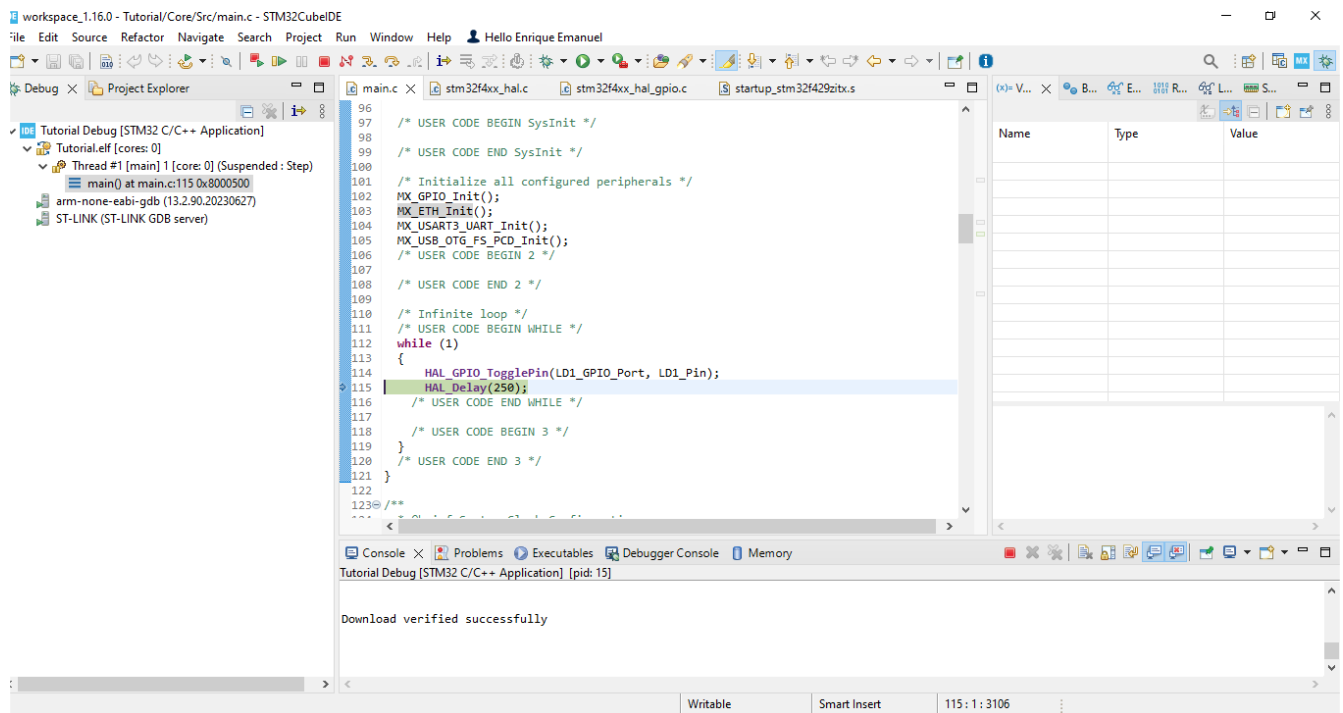
Durante este proceso, se abrirá una ventana que mostrará el valor de cada variable en tiempo real a lo largo de la ejecución del programa.

Para ejecutar las instrucciones paso a paso, presiona la tecla F6.

Antes de ejecutar la instrucción `HAL_GPIO_TogglePin` por primera vez, el LED permanecerá apagado.



Luego al ejecutar la instrucci3n el led en la tarjeta deber3a cambiar de estado a Encendido.



Enlaces

Repositorio del Grupo 1:

https://github.com/Dariuss-ing-electronica/Grupo_1_TDII_2025

APP1.1:

https://github.com/Dariuss-ing-electronica/Grupo_1_TDII_2025/tree/main/AFP_1_TDII_2025/App_1_4_Grupo_1_2025

APP1.2:

https://github.com/Dariuss-ing-electronica/Grupo_1_TDII_2025/tree/main/AFP_1_TDII_2025/App_1_2_Grupo_1_2025

APP1.3:

https://github.com/Dariuss-ing-electronica/Grupo_1_TDII_2025/tree/main/AFP_1_TDII_2025/App_1_3_Grupo_%201_%202025

APP1.4:

https://github.com/Dariuss-ing-electronica/Grupo_1_TDII_2025/tree/main/AFP_1_TDII_2025/App_1_4_Grupo_1_2025